UNIT-III

INTERMEDIATE CODE GENERATION

In Intermediate code generation we use syntax directed methods to translate the source program into an intermediate form programming language constructs such as declarations, assignments and flow-of-control statements.

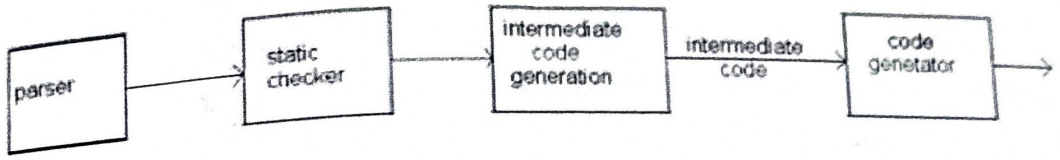


Figure 4.1 : Intermediate Code Generator

Intermediate code is:

- The output of the Parser and the input to the Code Generator.
- Relatively machine-independent and allows the compiler to be retargeted.
- Relatively easy to manipulate (optimize).

What are the Advantages of an intermediate language?

Advantages of Using an Intermediate Language includes :

1. Retargeting is facilitated - Build a compiler for a new machine by attaching a new code generator to an existing front-end.

2. Optimization - reuse intermediate code optimizers in compilers for different languages and different machines.

Note: the terms "intermediate code", "intermediate language", and "intermediate representation" are all used interchangeably.

Types of Intermediate representations / forms: There are three types of intermediate representation:-

1. Syntax Trees
2. Postfix notation
3. Three Address Code

Semantic rules for generating three-address code from common programming language constructs are similar to those for constructing syntax trees of for generating postfix notation.

Graphical Representations

A syntax tree depicts the natural hierarchical structure of a source program. A DAG (Directed Acyclic Graph) gives the same information but in a more compact way because common sub-expressions are identified. A syntax tree for the assignment statement a:=b*-c+b*-c appear in the following figure.

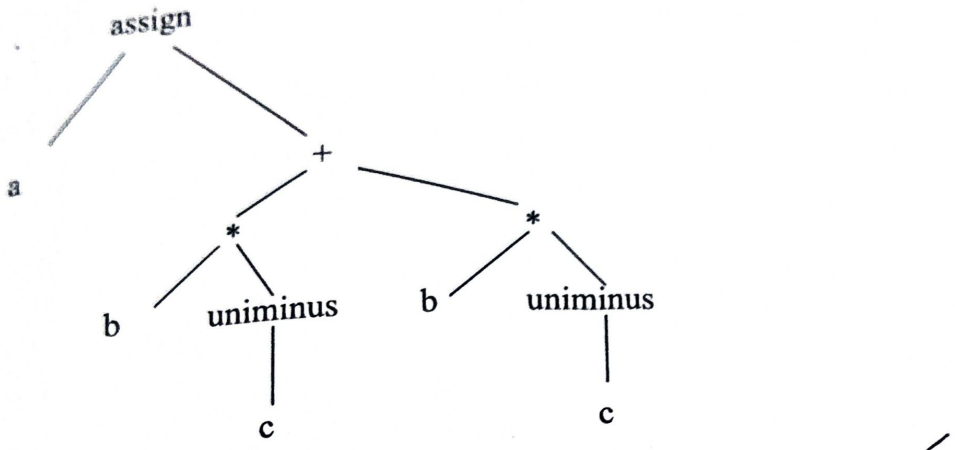


Figure 4.2 : Abstract Syntax Tree for the statement a:=b*-c+b*-c

Postfix notation is a linearized representation of a syntax tree; it is a list of the nodes of the in which a node appears immediately after its children. The postfix notation for the syntax tree in the fig is

a b c **uminus** + b c **uminus** * + **assign**

The edges in a syntax tree do not appear explicitly in postfix notation. They can be recovered in the order in which the nodes appear and the no. of operands that the operator at a node expects. The recovery of edges is similar to the evaluation, using a staff, of an expression in postfix notation.

What is Three Address Code?

Three-address code is a sequence of statements of the general form : X := Y Op Z

where x, y, and z are names, constants, or compiler-generated temporaries; op stands for any operator, such as a fixed- or floating-point arithmetic operator, or a logical operator on Boolean-valued data. Note that no built-up arithmetic expressions are permitted, as there is only one operator on the right side of a statement. Thus a source language expression like x+y*z might be translated into a sequence

```
t1 := y * z
t2 := x + t1
```

Where t1 and t2 are compiler-generated temporary names. This unraveling of complicated arithmetic expressions and of nested flow-of-control statements makes three-address code desirable for target code generation and optimization. The use of names for the intermediate values computed by a program allow- three-address code to be easily rearranged – unlike postfix notation. Three - address code is a linearzed representation of a syntax tree or a dag in which explicit names correspond to the interior nodes of the graph.

Intermediate code using Syntax for the above arithmetic expression

```
t1 := -c
t2 := b * t1
t3 := -c
t4 := b * t3
t5 := t2 + t4
a := t5
```

The reason for the term"three-address code" is that each statement usually contains three addresses, two for the operands and one for the result. In the implementations of three-address code given later in this section, a programmer-defined name is replaced by a pointer tc a symbol-table entry for that name.

Types of Three-Address Statements

Three-address statements are akin to assembly code. Statements can have symbolic labels and there are statements for flow of control. A symbolic label represents the index of a three-address statement in the array holding inter- mediate code. Actual indices can be substituted for the labels either by making a separate pass, or by using "back patching," discussed in Section 8.6. Here are the common three-address statements used in the remainder of this book:

1. **Assignment statements** of the form x: = y op z, where op is a binary arithmetic or logical operation.

2. **Assignment instructions** of the form x:= op y, where op is a unary operation. Essential unary operations include unary minus, logical negation, shift operators, and conversion operators that, for example, convert a fixed-point number to a floating-point number.

3. **Copy statements** of the form x: = y where the value of y is assigned to x.

4. **The unconditional jump** goto L. The three-address statement with label L is the next to be executed.

5. **Conditional jumps** such as if x relop y goto L. This instruction applies a relational operator (<, =, >=, etc.) to x and y, and executes the statement with label L next if x stands in relation relop to y. If not, the three-address statement following if x relop y goto L is executed next, as in the usual sequence.

6. **param x and call p, n** for procedure calls and return y, where y representing a returned value is optional. Their typical use is as the sequence of three-address statements

param x1
param x2
param xn
call p, n

Generated as part of a call of the procedure p(x,, x~,..., x"). The integer n indicating the number of actual parameters in "call p, n" is not redundant because calls can be nested. The implementation of procedure calls is outline d in Section 8.7.

7. **Indexed assignments** of the form x: = y[i] and x [i]: = y. The first of these sets x to the value in the location i memory units beyond location y. The statement x[i]:=y sets the contents of the location i units beyond x to the value of y. In both these instructions, x, y, and i refer to data objects.

8. **Address and pointer assignments** of the form x:= &y, x:= *y and *x: = y. The first of these sets the value of x to be the location of y. Presumably y is a name, perhaps a temporary, that denotes an expression with an I-value such as A[i, j], and x is a pointer name or temporary. That is, the r-value of x is the l-value (location) of some object!. In the statement x: = ~y, presumably y is a pointer or a temporary whose r- value is a location. The r-value of x is made equal to the contents of that location. Finally, +x: = y sets the r-value of the object pointed to by x to the r-value of y.

The choice of allowable operators is an important issue in the design of an intermediate form. The operator set must clearly be rich enough to implement the operations in the source language. A small operator set is easier to implement on a new target machine. However, a restricted instruction set may force the front end to generate long sequences of statements for some source, language operations. The optimizer and code generator may then have to work harder if good code is to be generated.

SYNTAX DIRECTED TRANSLATION OF THREE ADDRESS CODE

When three-address code is generated, temporary names are made up for the interior nodes of a syntax tree. The value of non- terminal E on the left side of E □ E1 + E will be

computed into a new temporary t. In general, the three- address code for id: = E consists of code to evaluate E into some temporary t, followed by the assignment id.place: = t. If an expression is a single identifier, say y, then y itself holds the value of the expression. For the moment, we create a new name every time a temporary is needed; techniques for reusing temporaries are given in Section S.3. The S-attributed definition in Fig. 8.6 generates three-address code for assignment statements. Given input a: = b+ – c + b+ – c, it produces the code in Fig. 8.5(a). The synthesized attribute S.code represents the three- address code for the assignment S. The non-terminal E has two attributes:

1. E.place, the name that will hold the value of E, and

2. E.code, the sequence of three-address statements evaluating E.

The function newtemp returns a sequence of distinct names t1, t2,... in response to successive calls. For convenience, we use the notation gen(x ': =' y '+' z) in Fig. 8.6 to represent the three-address statement x: = y + z. Expressions appearing instead of variables like x, y, and z are evaluated when passed to gen, and quoted operators or operands, like '+', are taken literally. In practice, three- address statements might be sent to an output file, rather than built up into the code attributes. Flow-of-control statements can be added to the language of assignments in Fig. 8.6 by productions and semantic rules)like the ones for while statements in Fig. 8.7. In the figure, the code for S - while E do S, is generated using' new attributes S.begin and S.after to mark the first statement in the code for E and the statement following the code for S, respectively.

PRODUCTION	SEMANTIC RULES
$S \rightarrow id := E$	$S.code := E.code \parallel gen(id.place ':=' E.place)$
$E \rightarrow E_1 + E_2$	$E.place := newtemp;$ $E.code := E_1.code \parallel E_2.code \parallel gen(E.place ':=' E_1.place '+' E_2.place)$
$E \rightarrow E_1 * E_2$	$E.place := newtemp;$ $E.code := E_1.code \parallel E_2.code \parallel gen(E.place ':=' E_1.place '*' E_2.place)$
$E \rightarrow - E_1$	$E.place := newtemp;$ $E.code_1 := E_1.code \parallel gen(E.place ':=' 'uminus' E_1.place)$
$E \rightarrow (E_1)$	$E.place := E_1.place;$ $E.code := E_1.code$
$E \rightarrow id$	$E.place := id.place;$ $E.code := ''$

These attributes represent labels created by a function new label that returns a new label every time it is called.

IMPLEMENTATIONS OF THREE-ADDRESS STATEMENTS:

A three-address statement is an abstract form of intermediate code. In a compiler, these statements can be implemented as records with fields for the operator and the operands. Three such representations are quadruples, triples, and indirect triples.

QUADRUPLES:

A quadruple is a record structure with four fields, which we call op, arg l, arg 2, and result. The op field contains an internal code for the operator. The three-address statement x:= y op z is represented by placing y in arg 1. z in arg 2. and x in result. Statements with unary operators like x: = – y or x: = y do not use arg 2. Operators like param use neither arg2 nor result. Conditional and unconditional jumps put the target label in result. The quadruples in Fig. H.S(a) are for the assignment a: = b+ – c + b i – c. They are obtained from the three-address code .The contents of fields arg 1, arg 2, and result are normally pointers to the symbol-table entries for the names represented by these fields. If so, temporary names must be entered into the symbol table as they are created.

TRIPLES:

To avoid entering temporary names into the symbol table. We might refer to a temporary value bi the position of the statement that computes it. If we do so, three-address statements can be represented by records with only three fields: op, arg 1 and arg2, as Shown below. The fields arg l and arg2, for the arguments of op, are either pointers to the symbol table (for programmer-defined names or constants) or pointers into the triple structure (for temporary values). Since three fields are used, this intermediate code format is known as triples.' Except for the treatment of programmer-defined names, triples correspond to the representation of a syntax tree or dag by an array of nodes, as in

	op	Arg1	Arg2	Result
(0)	uminus	c		t1
(1)	*	b	t1	t2
(2)	uminus	c		t3
(3)	*	b	t3	t4
(4)	+	t2	t4	t5
(5)	:=	t5		A

Table 8.8 (a) : Qudraples

	op	Arg1	Arg2
(0)	uminus	C	
(1)	*	B	(0)
(2)	uminus	C	
(3)	*	B	(2)
(4)	+	(1)	(3)
(5)	:=	A	(4)

Table8.8(b) : Triples :Triples

Parenthesized numbers represent pointers into the triple structure, while symbol-table pointers are represented by the names themselves. In practice, the information needed to interpret the different kinds of entries in the arg 1 and arg2 fields can be encoded into the op field or some additional fields. The triples in Fig. 8.8(b) correspond to the quadruples in Fig. 8.8(a). Note that

the copy statement a:= t5 is encoded in the triple representation by placing a in the arg 1 field and using the operator assign. A ternary operation like x[i]: = y requires two entries in the triple structure, as shown in Fig. 8.9(a), while x: = y[i] is naturally represented as two operations in Fig. 8.9(b).

	op	arg 1	arg 2
(0)	[ ]=	x	i
(1)	assign	(0)	y

(a) x[i] := y

	op	arg 1	arg 2
(0)	=[ ]	y	i
(1)	assign	x	(0)

(b) x := y[i]

Fig. 8.9. More triple representations.

Indirect Triples

Another implementation of three-address code that has been considered is that of listing pointers to triples, rather than listing the triples themselves. This implementation is naturally called indirect triples. For example, let us use an array statement to list pointers to triples in the desired order. Then the triples in Fig. 8.8(b) might be represented as in Fig. 8.10.

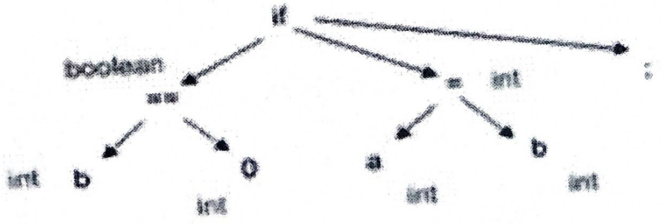
	statement
(0)	(14)
(1)	(15)
(2)	(16)
(3)	(17)
(4)	(18)
(5)	(19)

	op	arg 1	arg 2
(14)	uminus	c	
(15)	*	b	(14)
(16)	uminus	c	
(17)	*	b	(16)
(18)	+	(15)	(17)
(19)	assign	a	(18)

Figure 8.10 : Indirect Triples

SEMANTIC ANALYSIS : This phase focuses mainly on the

. Checking the semantics ,
. Error reporting
. Disambiguate overloaded operators
. Type coercion ,
. Static checking



- Type checking
-Control flow checking
- Uniqueness checking
- Name checking aspects of translation

Assume that the program has been verified to be syntactically correct and converted into some kind of intermediate representation (a parse tree). One now has parse tree available. The next phase will be semantic analysis of the generated parse tree. Semantic analysis also includes error reporting in case any semantic error is found out.

Semantic analysis is a pass by a compiler that adds semantic information to the parse tree and performs certain checks based on this information. It logically follows the parsing phase, in which the parse tree is generated, and logically precedes the code generation phase, in which (intermediate/target) code is generated. (In a compiler implementation, it may be possible to fold different phases into one pass.) Typical examples of semantic information that is added and checked is typing information (type checking) and the binding of variables and function names to their definitions (object binding). Sometimes also some early code optimization is done in this phase. For this phase the compiler usually maintains symbol tables in which it stores what each symbol (variable names, function names, etc.) refers to.

FOLLOWING THINGS ARE DONE IN SEMANTIC ANALYSIS:

Disambiguate Overloaded operators: If an operator is overloaded, one would like to specify the meaning of that particular operator because from one will go into code generation phase next.

TYPE CHECKING: The process of verifying and enforcing the constraints of types is called type checking. This may occur either at compile-time (a static check) or run-time (a dynamic check). Static type checking is a primary task of the semantic analysis carried out by a compiler. If type rules are enforced strongly (that is, generally allowing only those automatic type conversions which do not lose information), the process is called strongly typed, if not, weakly typed.

UNIQUENESS CHECKING: Whether a variable name is unique or not, in the its scope.

Type coersion: If some kind of mixing of types is allowed. Done in languages which are not strongly typed. This can be done dynamically as well as statically.

NAME CHECKS: Check whether any variable has a name which is not allowed. Ex. Name is same as an identifier (Ex, int in java).

- Parser cannot catch all the program errors
- There is a level of correctness that is deeper than syntax analysis
- Some language features cannot be modeled using context free grammar formalism

- Whether an identifier has been declared before use, this problem is of identifying a language $\{w \alpha w \mid w \varepsilon \Sigma^*\}$

- This language is not context free

A parser has its own limitations in catching program errors related to semantics, something that is deeper than syntax analysis. Typical features of semantic analysis cannot be modeled using context free grammar formalism. If one tries to incorporate those features in the definition of a language then that language doesn't remain context free anymore.

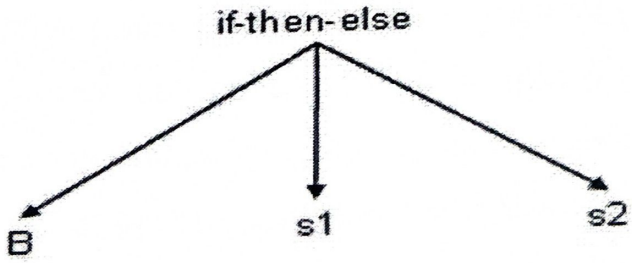
Example: in

```
string x; int y;
y = x + 3        the use of x is a type error
int a, b;
a = b + c        c is not declared
```

An identifier may refer to different variables in different parts of the program . An identifier may be usable in one part of the program but not another These are a couple of examples which tell us that typically what a compiler has to do beyond syntax analysis. The third point can be explained like this: An identifier x can be declared in two separate functions in the program, once of the type int and then of the type char. Hence the same identifier will have to be bound to these two different properties in the two different contexts. The fourth point can be explained in this manner: A variable declared within one function cannot be used within the scope of the definition of the other function unless declared there separately. This is just an example. Probably you can think of many more examples in which a variable declared in one scope cannot be used in another scope.

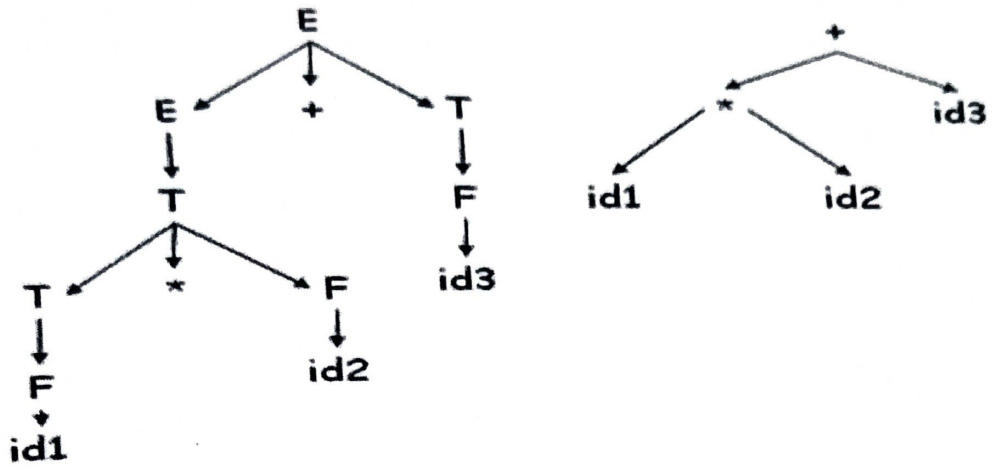
ABSTRACT SYNTAX TREE: Is nothing but the condensed form of a parse tree, It is

- Σ Useful for representing language constructs so naturally.
- Σ The production S ⟶ if B then s1 else s2 may appear as



the next few slides we will see how abstract syntax trees can be constructed from syntax ected definitions. Abstract syntax trees are condensed form of parse trees. Normally operators d keywords appear as leaves but in an abstract syntax tree they are associated with the interior des that would be the parent of those leaves in the parse tree. This is clearly indicated by the amples in these slides.

Chain of single productions may be collapsed, and operators move to the parent nodes



Chain of single productions are collapsed into one node with the operators moving up to become the node.

CONSTRUCTING ABSTRACT SYNTAX TREE FOR EXPRESSIONS:

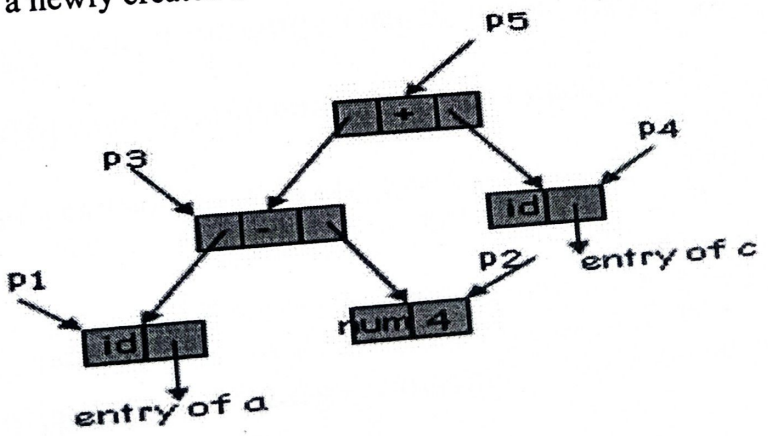
In constructing the Syntax Tree, we follow the convention that :

. Each node of the tree can be represented as a record consisting of at least two fields to store operators and operands.

.operators **:** one field for operator, remaining fields ptrs to operands mknode(op,left,right)
.identifier **:** one field with label id and another ptr to symbol table mkleaf(id, id.entry)
.number **:** one field with label num and another to keep the value of the number mkleaf(num,val)

Each node in an abstract syntax tree can be implemented as a record with several fields. In the node for an operator one field identifies the operator (called the label of the node) and the remaining contain pointers to the nodes for operands. Nodes of an abstract syntax tree may have additional fields to hold values (or pointers to values) of attributes attached to the node. The functions given in the slide are used to create the nodes of abstract syntax trees for expressions. Each function returns a pointer to a newly created note.

For Example: the following sequence of function calls creates a parse tree for **w= a- 4 + c**



P 1 = mkleaf(id, entry.a)
P 2 = mkleaf(num, 4)
P 3 = mknode(-, P 1 , P 2)
P 4 = mkleaf(id, entry.c)

P 5 = mknode(+, P 3 , P 4)

An example showing the formation of an abstract syntax tree by the given function calls for the expression a-4+c.The call sequence can be defined based on its postfix form, which is explained blow.

A- Write the postfix equivalent of the expression for which we want to construct a syntax tree

For above string w=a-4+c, it is **a4-c+**

B- Call the functions in the sequence, as defined by the sequence in the postfix expression which results in the desired tree. In the case above, call mkleaf() for a, mkleaf() for 4, mknode() for -, mkleaf() for c , and mknode() for + at last.

1. P1 = **mkleaf**(id, a.entry) : A leaf node made for the identifier a, and an entry for a is made in the symbol table.

2. P2 = **mkleaf**(num,4) : A leaf node made for the number 4, and entry for its value.

3. P3 = **mknode**(-,P1,P2) : An internal node for the -, takes the pointer to previously made nodes P1, P2 as arguments and represents the expression a-4.

4. P4 = **mkleaf**(id, c.entry) : A leaf node made for the identifier c , and an entry for c.entry made in the symbol table.

5. P5 = **mknode**(+,P3,P4) : An internal node for the + , takes the pointer to previously made nodes P3,P4 as arguments and represents the expression a- 4+c .

Following is the syntax directed definition for constructing syntax tree above

E ⟶E 1 + T	E.ptr = mknode(+, E1 .ptr, T.ptr)
E⟶ T	E.ptr = T.ptr
T ⟶T 1 * F	T.ptr := mknode(*, T1 .ptr, F.ptr)
T ⟶F	T.ptr := F.ptr
F ⟶(E)	F.ptr := E.ptr
F ⟶id	F.ptr := mkleaf(id, id.entry)
F⟶ num	F.ptr := mkleaf(num, val)

Now we have the syntax directed definitions to construct the parse tree for a given grammar. All the rules mentioned in slide 29 are taken care of and an abstract syntax tree is formed.

ATTRIBUTE GRAMMARS: A CFG G=(V,T,P,S), is called an Attributed Grammar iff , where in G, each grammar symbol XЄ VUT, has an associated set of attributes, and each production, p ЄP, is associated with a set of attribute evaluation rules called Semantic Actions.

In an **AG**, the values of attributes at a parse tree node are computed by semantic rules. There are two different specifications of **AGs** used by the **Semantic** Analyzer in evaluating the semantics of the program constructs. They are,

- **Syntax directed definition(SDD)s**
  - High level specifications
  - Hides implementation details
  - Explicit order of evaluation is not specified
- **Syntax directed Translation schemes(SDT)s**
  - Nothing but an SDD, which indicates order in which semantic rules are to be evaluated and
  - Allow some implementation details to be shown.

An **attribute grammar** is the formal expression of the syntax-derived semantic checks associated with a grammar. It represents the rules of a language not explicitly imparted by the syntax. In a practical way, it defines the information that is needed in the abstract syntax tree in order to successfully perform semantic analysis. This information is stored as attributes of the nodes of the abstract syntax tree. The values of those attributes are calculated by semantic rule.

There are two ways for writing attributes:

1) **Syntax Directed Definition(SDD):** Is a context free grammar in which a set of semantic actions are embedded (associated) with each production of G.

It is a high level specification in which implementation details are hidden, e.g., S.sys = A.sys + B.sys;

/* does not give any implementation details. It just tells us. This kind of attribute equation we will be using, Details like at what point of time is it evaluated and in what manner are hidden from the programmer.*/

E ⟶E1 + T	{ E.val = E1 .val+ E2.val }
E⟶ T	{ E.val = T.val }
T ⟶T 1 * F	{ T.val = T1 .val+ F.val)
T ⟶F	{ T.val = F.val }
F ⟶(E)	{ F.val = E.val }
F ⟶id	{ F.val = id.lexval }
F⟶ num	{ F.val = num.lexval }

2) **Syntax directed Translation(SDT) scheme**: Sometimes we want to control the way th attributes are evaluated, the order and place where they are evaluated. This is of a slightly lowe level.

An SDT is an SDD in which semantic actions can be placed at any position in the body of the production.

For example , following SDT prints the prefix equivalent of an arithmetic expression consisting a + and * operators.

L⟶ En{ printf('E.val') }
E ⟶ { printf('+') }E1 + T
E⟶ T
T ⟶ { printf('*') }T 1 * F
T ⟶ F
F ⟶ (E)
F ⟶ { printf('id.lexval') }id
F⟶ { printf('num.lexval') } num

This action in an SDT, is executed as soon as its node in the parse tree is visited in a preorder traversal of the tree.

Conceptually both the SDD and SDT schemes will:

- Parse input token stream
- Build parse tree
- Traverse the parse tree to evaluate the semantic rules at the parse tree nodes

Evaluation may:

- Generate code
- Save information in the symbol table
- Issue error messages
- Perform any other activity

To avoid repeated traversal of the parse tree, actions are taken simultaneously when a token is found. So calculation of attributes goes along with the construction of the parse tree.

Along with the evaluation of the semantic rules the compiler may simultaneously generate code, save the information in the symbol table, and/or issue error messages etc. at the same time while building the parse tree.

This saves multiple passes of the parse tree.

Example

Number ⟶ sign list
sign ⟶ + | -
list ⟶ list bit | bit
bit ⟶ 0 | 1

Build attribute grammar that annotates Number with the value it represents

. Associate attributes with grammar symbols

symbol	attributes
Number	value
sign	negative
list	position, value
bit	position, value

production Attribute rule number ⟶sign list

list.position ⟵ 0

if sign.negative

then number.value ⟵ - list.value

else number.value ⟵ list.value

sign ⟶+ sign.negative ⟵ false sign ⟶- sign.negative ⟵ true list ⟶bit

bit.position ⟵ list.position

list.value ⟵ bit.value

$list_0$ ⟶$list_1$ bit

$list_1$ .position ⟵ $list_0$ .position + 1

bit.position ⟵ $list_0$ .position

$list_0$ .value ⟵ $list_1$ .value + bit.value

bit ⟶0 bit.value ⟵ 0 bit ⟶1 bit.value ⟵ $2^{bit.position}$

Explanation of attribute rules

Num -> sign list /*since list is the rightmost so it is assigned position 0
*Sign determines whether the value of the number would be
same or the negative of the value of list/

Sign -> + | - /*Set the Boolean attribute (negative) for sign*/

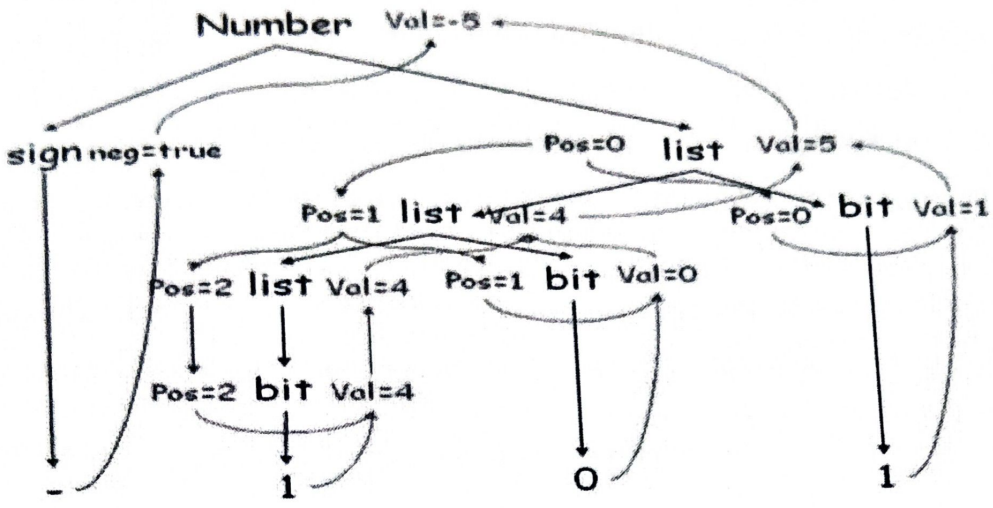
List -> bit /*bit position is the same as list position because this bit is the rightmost
value of the list is same as bit./

List0 -> List1 bit /*position and value calculations*/

Bit -> 0 | 1 /*set the corresponding value*/

Attributes of RHS can be computed from attributes of LHS and vice versa.

The Parse Tree and the Dependence graph are as under



Dependence graph shows the dependence of attributes on other attributes, along with the syntax tree. Top down traversal is followed by a bottom up traversal to resolve the dependencies. Number, val and neg are synthesized attributes. Pos is an inherited attribute.

Attributes : . Attributes fall into two classes namely *synthesized attributes* and *inherited attributes*. Value of a synthesized attribute is computed from the values of its children nodes. Value of an inherited attribute is computed from the sibling and parent nodes.

The attributes are divided into two groups, called synthesized attributes and inherited attributes. The synthesized attributes are the result of the attribute evaluation rules also using the values of the inherited attributes. The values of the inherited attributes are inherited from parent nodes and siblings.

Each grammar production A ⟶a has associated with it a set of semantic rules of the form

$b = f(c_1, c_2, ..., c_k)$, Where f is a function, and either ,b is a synthesized attribute of A Or

- b is an inherited attribute of one of the grammar symbols on the right

. attribute b depends on attributes $c_1, c_2, ..., c_k$

Dependence relation tells us what attributes we need to know before hand to calculate a particular attribute.

Here the value of the attribute b depends on the values of the attributes c_1 to c_k . If c_1 to c_k belong to the children nodes and b to A then b will be called a synthesized attribute. And if b belongs to one among a (child nodes) then it is an inherited attribute of one of the grammar symbols on the right.

Synthesized Attributes : A syntax directed definition that uses only synthesized attributes is said to be an S- attributed definition

. A parse tree for an S-attributed definition can be annotated by evaluating semantic rules for attributes

S-attributed grammars are a class of attribute grammars, comparable with L-attributed grammars but characterized by having no inherited attributes at all. Inherited attributes, which must be passed down from parent nodes to children nodes of the abstract syntax tree during the semantic analysis, pose a problem for bottom-up parsing because in bottom-up parsing, the parent nodes of the abstract syntax tree are created *after* creation of all of their children. Attribute evaluation in S-attributed grammars can be incorporated conveniently in both top-down parsing and bottom-up parsing .

Syntax Directed Definitions for a desk calculator program

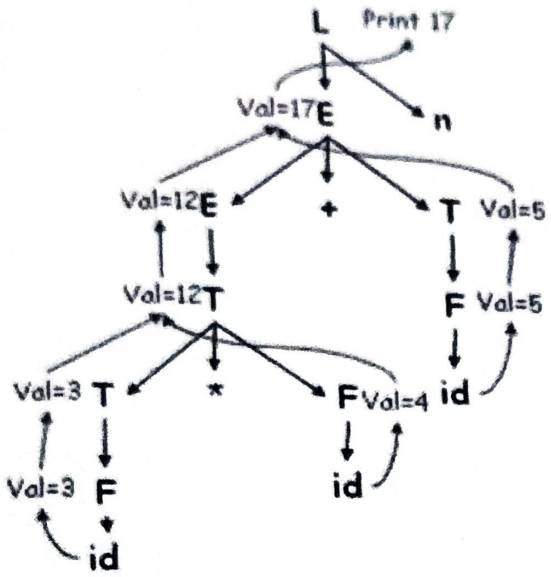
L ⟶ E n	Print (E.val)
E ⟶ E + T	E.val = E.val + T.val
E ⟶ T	E.val = T.val
T ⟶ T * F	T.val = T.val * F.val
T ⟶ F	T.val = F.val
F ⟶ (E)	F.val = E.val
F ⟶ digit	F.val = digit.lexval

. terminals are assumed to have only synthesized attribute values of which are supplied by lexical analyzer

. start symbol does not have any inherited attribute

This is a grammar which uses only synthesized attributes. Start symbol has no parents, hence no inherited attributes.

Parse tree for 3 * 4 + 5 n



Using the previous attribute grammar calculations have been worked out here for 3 * 4 + 5 n. Bottom up parsing has been done.

Inherited Attributes: An inherited attribute is one whose value is defined in terms of attributes at the parent and/or siblings

. Used for finding out the context in which it appears

. possible to use only S-attributes but more natural to use inherited attributes

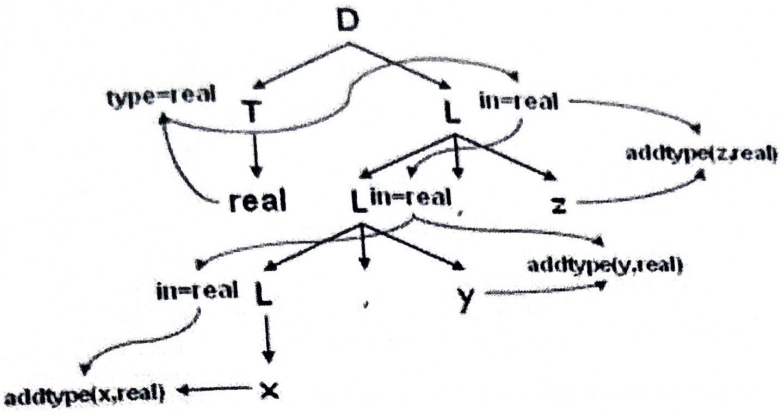
D ⟶ T L	L.in = T.type
T ⟶ real	T.type = real
T ⟶ int	T.type = int
L ⟶ L_1 , id	L_1 .in = L.in; addtype(id.entry, L.in)
L ⟶ id	addtype (id.entry,L.in)

Inherited attributes help to find the context (type, scope etc.) of a token e.g., the type of a token or scope when the same variable name is used multiple times in a program in different functions. An inherited attribute system may be replaced by an S -attribute system but it is more natural to use inherited attributes in some cases like the example given above.

Here addtype(a, b) functions adds a symbol table entry for the id a and attaches to it the type of b

.

Parse tree for real x, y, z



Dependence of attributes in an inherited attribute system. The value of in (an inherited attribute) at the three L nodes gives the type of the three identifiers x , y and z . These are determined by computing the value of the attribute T.type at the left child of the root and then valuating L.in top down at the three L nodes in the right subtree of the root. At each L node the procedure addtype is called which inserts the type of the identifier to its entry in the symbol table. The figure also shows the dependence graph which is introduced later.

Dependence Graph : . If an attribute b depends on an attribute c then the semantic rule for b must be evaluated after the semantic rule for c

. The dependencies among the nodes can be depicted by a directed graph called dependency graph

Dependency Graph **:** Directed graph indicating interdependencies among the synthesized and inherited attributes of various nodes in a parse tree.

Algorithm to construct dependency graph

for each node **n** in the parse tree do

for each attribute **a** of the grammar symbol do

construct a node in the dependency graph

for **a**

for each node n in the parse tree do

for each semantic rule $b = f(c_1, c_2, \ldots, c_k)$ do

{ associated with production at n }

for i = 1 to k do

Construct an edge from c_i to b

An algorithm to construct the dependency graph. After making one node for every attribute of all the nodes of the parse tree, make one edge from each of the other attributes on which it depends.

For example ,

- **Suppose A.a = f(X.x , Y.y) is a semantic rule for A →X Y**

- **If production A →X Y has the semantic rule X.x = g(A.a, Y.y)**

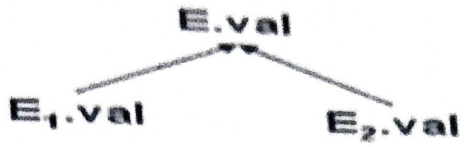
The semantic rule A.a = f(X.x , Y.y) for the production A -> XY defines the synthesized attribute a of A to be dependent on the attribute x of X and the attribute y of Y . Thus the dependency graph will contain an edge from X.x to A.a and Y.y to A.a accounting for the two dependencies. Similarly for the semantic rule X.x = g(A.a , Y.y) for the same production there will be an edge from A.a to X.x and an edg e from Y.y to X.x.

Example

. Whenever following production is used in a parse tree

$E \longrightarrow E\,1 + E\,2$ E.val = E 1 .val + E 2 .val

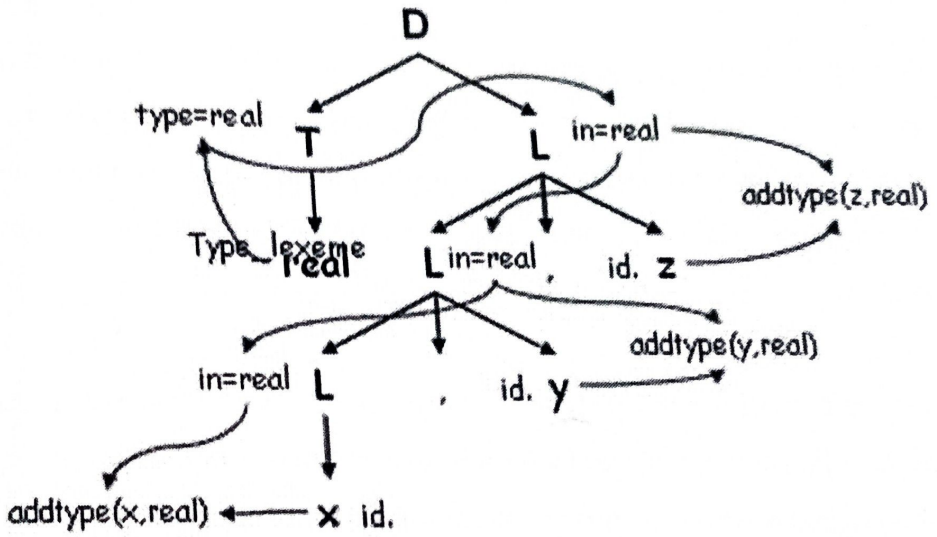
we create a dependency graph



The synthesized attribute E.val depends on E1.val and E2.val hence the two edges one each from E 1 .val & E 2 .val

For example, the dependency graph for the sting **real** id1, id2, id3

. Put a dummy synthesized attribute b for a semantic rule that consists of a procedure call



The figure shows the dependency graph for the statement real id1, id2, id3 along with the parse tree. Procedure calls can be thought of as rules defining the values of dummy synthesized attributes of the nonterminal on the left side of the associated production. Blue arrows constitute the dependency graph and black lines, the parse tree. Each of the semantic rules addtype (id.entry, L.in) associated with the L productions leads to the creation of the dummy attribute.

Evaluation Order :

Any topological sort of dependency graph gives a valid order in which semantic rules must be evaluated

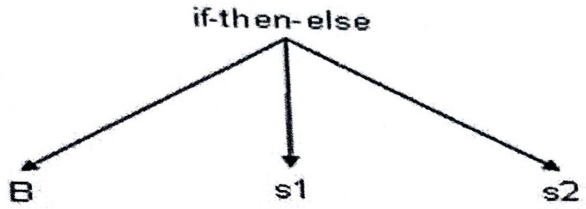
a4 = real
a5 = a4
addtype(id3.entry, a5)
a7 = a5
addtype(id2.entry, a7)

a9 := a7 addtype(id1.entry, a9)

A topological sort of a directed acyclic graph is any ordering m1, m2, m3 .....mk of the nodes of the graph such that edges go from nodes earlier in the ordering to later nodes. Thus if mi -> mj is an edge from mi to mj then mi appears before mj in the ordering. The order of the statements shown in the slide is obtained from the topological sort of the dependency graph in the previous slide. 'an' stands for the attribute associated with the node numbered n in the dependency graph. The numbering is as shown in the previous slide.

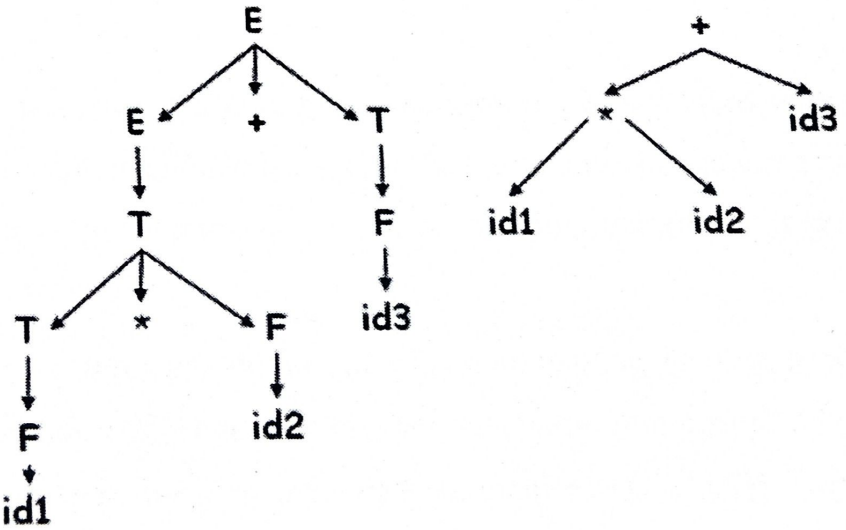
Abstract Syntax Tree is the condensed form of the parse tree, which is

. Useful for representing language constructs.
. The production : S ⟶ **if** B **then** s1 else s2 may appear as



In the next few slides we will see how abstract syntax trees can be constructed from syntax directed definitions. Abstract syntax trees are condensed form of parse trees. Normally operators and keywords appear as leaves but in an abstract syntax tree they are associated with the interior nodes that would be the parent of those leaves in the parse tree. This is clearly indicated by the examples in these slides.

. Chain of single productions may be collapsed, and operators move to the parent nodes



Chain of single production are collapsed into one node with the operators moving up to become the node.

For Constructing the Abstract Syntax tree for expressions,

. Each node can be represented as a record

. *operators* : one field for operator, remaining fields ptrs to operands mknode(op,left,right)

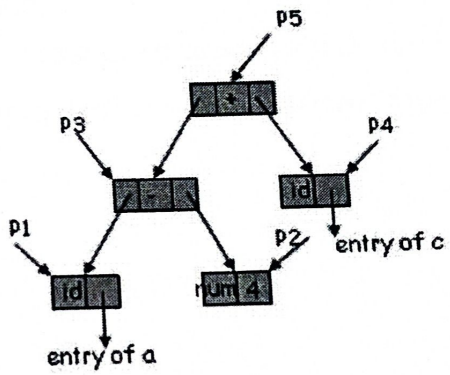
. *identifier* : one field with label id and another ptr to symbol table mkleaf(id,entry)

. *number* : one field with label num and another to keep the value of the number mkleaf(num,val)

Each node in an abstract syntax tree can be implemented as a record with several fields. In the node for an operator one field identifies the operator (called the label of the node) and the remaining contain pointers to the nodes for operands. Nodes of an abstract syntax tree may have additional fields to hold values (or pointers to values) of attributes attached to the node. The functions given in the slide are used to create the nodes of abstract syntax trees for expressions. Each function returns a pointer to a newly created note.

Example : **The following sequence of function calls creates a parse tree for a- 4 + c**

P 1 = mkleaf(id, entry.a)
P 2 = mkleaf(num, 4)
P 3 = mknode(-, P 1 , P 2)
P 4 = mkleaf(id, entry.c)
P 5 = mknode(+, P 3 , P 4)



An example showing the formation of an abstract syntax tree by the given function calls for the expression a-4+c.The call sequence can be explained as:

1. P1 = mkleaf(id,entry.a) : A leaf node made for the identifier Qa R and an entry for Qa R is made in the symbol table.
2. P2 = mkleaf(num,4) : A leaf node made for the number Q4 R.
3. P3 = mknode(-,P1,P2) : An internal node for the Q- Q.I takes the previously made nodes as arguments and represents the expression Qa-4 R.
4. P4 = mkleaf(id,entry.c) : A leaf node made for the identifier Qc R and an entry for Qc R is made in the symbol table.
5. P5 = mknode(+,P3,P4) : An internal node for the Q+ Q.I takes the previously made nodes as arguments and represents the expression Qa- 4+c R.

A syntax directed definition for constructing syntax tree

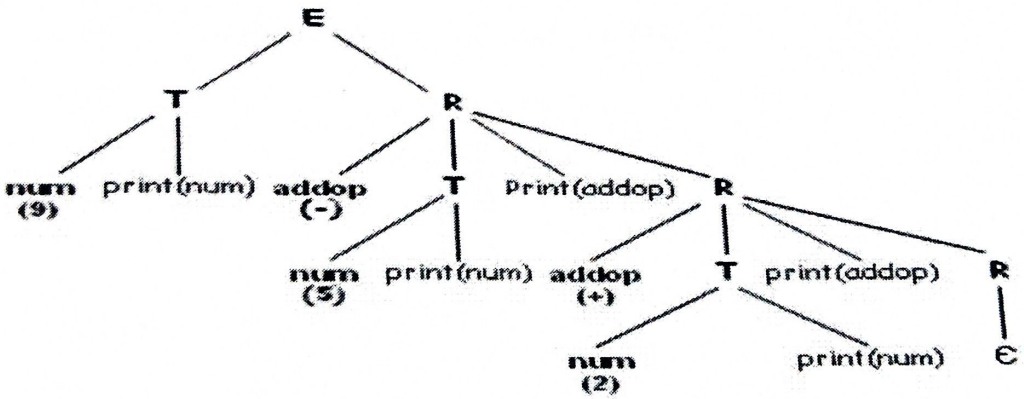
E → E 1 + T	E.ptr = mknode(+, E 1 .ptr, T.ptr)
E → T	E.ptr = T.ptr
T → T 1 * F	T.ptr := mknode(*, T 1 .ptr, F.ptr)
T → F	T.ptr := F.ptr
F → (E)	F.ptr := E.ptr
F → id	F.ptr := mkleaf(id, entry.id)
F → num	F.ptr := mkleaf(num,val)

Now we have the syntax directed definitions to construct the parse tree for a given grammar. All the rules mentioned in slide 29 are taken care of and an abstract syntax tree is formed.

Translation schemes : A CFG where semantic actions occur within the right hand side of production, A translation scheme to map infix to postfix.

E → T R
R→ addop T { print(addop)} R | e
T → num {print(num)}

Parse tree for 9 - 5 + 2



We assume that the actions are terminal symbols and Perform depth first order traversal to obtain 9 5 - 2 +.

- When designing translation scheme, ensure attribute value is available when referred to
- In case of synthesized attribute it is trivial (why?)

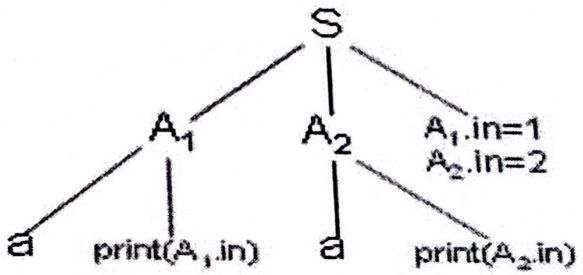
In a translation scheme, as we are dealing with implementation, we have to explicitly worry about the order of traversal. We can now put in between the rules some actions as part of the RHS. We put this rules in order to control the order of traversals. In the given example, we have two terminals (num and addop). It can generally be seen as a number followed by R (which

necessarily has to begin with an addop). The given grammar is in infix notation and we need to convert it into postfix notation. If we ignore all the actions, the parse tree is in black, without the red edges. If we include the red edges we get a parse tree with actions. The actions are so far treated as a terminal. Now, if we do a depth first traversal, and whenever we encounter a action we execute it, we get a post-fix notation. In translation scheme, we have to take care of the evaluation order; otherwise some of the parts may be left undefined. For different actions, different result will be obtained. Actions are something we write and we have to control it. Please note that translation scheme is different from a syntax driven definition. In the latter, we do not have any evaluation order; in this case we have an explicit evaluation order. By explicit evaluation order we have to set correct action at correct places, in order to get the desired output. Place of each action is very important. We have to find appropriate places, and that is that translation scheme is all about. If we talk of only synthesized attribute, the translation scheme is very trivial. This is because, when we reach we know that all the children must have been evaluated and all their attributes must have also been dealt with. This is because finding the place for evaluation is very simple, it is the rightmost place.

In case of both inherited and synthesized attributes

. An inherited attribute for a symbol on rhs of a production must be computed in an action before that symbol

S →A 1 A 2 {A 1 .in = 1,A 2 .in = 2}
A →a {print(A.in)}



Depth first order traversal gives error *undefined*

. A synthesized attribute for non terminal on the lhs can be computed after all attributes it references, have been computed. The action normally should be placed at the end of rhs

We have a problem when we have both synthesized as well as inherited attributes. For the given example, if we place the actions as shown, we cannot evaluate it. This is because, when doing a depth first traversal, we cannot print anything for A1. This is because A1 has not yet been initialized. We, therefore have to find the correct places for the actions. This can be that the inherited attribute of A must be calculated on its left. This can be seen logically from the definition of L-attribute definition, which says that when we reach a node, then everything on its left must have been computed. If we do this, we will always have the attribute evaluated at the

correct place. For such specific cases (like the given example) calculating anywhere on the left will work, but generally it must be calculated immediately at the left.

Example: Translation scheme for EQN

S →B	B.pts = 10
	S.ht = B.ht
B →B_1 B_2	B_1 .pts = B.pts
	B_2 .pts = B.pts
	B.ht = max(B_1 .ht,B_2 .ht)
B →B_1 sub B_2	B_1 .pts = B.pts;
	B_2 .pts = shrink(B.pts)
	B.ht = disp(B_1 .ht,B_2 .ht)
B →text	B.ht = text.h * B.pts

We now look at another example. This is the grammar for finding out how do I compose text. EQN was equation setting system which was used as an early type setting system for UNIX. It was earlier used as an latex equivalent for equations. We say that start symbol is a block: S - >B We can also have a subscript and superscript. Here, we look at subscript. A Block is composed of several blocks: B -> B1B2 and B2 is a subscript of B1. We have to determine what is the point size (inherited) and height Size (synthesized). We have the relevant function for height and point size given along side. After putting actions in the right place

$S \longrightarrow$ {B.pts = 10} B
{S.ht = B.ht}

$B \longrightarrow$ {B_1.pts = B.pts} B_1
{B_2.pts = B.pts} B_2
{B.ht = max(B_1.ht,B_2.ht)}

$B \longrightarrow$ {B_1.pts = B.pts} B_1 sub
{B_2.pts = shrink(B.pts)} B_2
{B.ht = disp(B_1.ht,B_2.ht)}

$B \longrightarrow$ text {B.ht = text.h * B.pts}

We have put all the actions at the correct places as per the rules stated. Read it from left to right and top to bottom. We note that all inherited attribute are calculated on the left of B symbols and synthesized attributes are on the right.

Top down Translation: Use predictive parsing to implement L-attributed definitions

E →E1 + T E.val := E 1 .val + T.val

E → E 1 - T	E.val := E 1 .val - T.val
E → T	E.val := T.val
T → (E)	T.val := E.val
T → num	T.val := num.lexval

We now come to implementation. We decide how we use parse tree and L-attribute definitions to construct the parse tree with a one-to-one correspondence. We first look at the top-down translation scheme. The first major problem is left recursion. If we remove left recursion by our standard mechanism, we introduce new symbols, and new symbols will not work with the existing actions. Also, we have to do the parsing in a single pass.

TYPE SYSTEM AND TYPE CHECKING:

. If both the operands of arithmetic operators +, -, x are integers then the result is of type integer
. The result of unary & operator is a pointer to the object referred to by the operand.
- If the type of operand is ***X*** then type of result is ***pointer to X***

In Pascal, types are classified under:

1. *Basic* types: These are atomic types with no internal structure. They include the types boolean, character, integer and real.

2. *Sub-range* types: A sub-range type defines a range of values within the range of another type. For example, type A = 1..10; B = 100..1000; U = 'A'..'Z';

3. *Enumerated* types: An enumerated type is defined by listing all of the possible values for the type. For example: type Colour = (Red, Yellow, Green); Country = (NZ, Aus, SL, WI, Pak, Ind, SA, Ken, Zim, Eng); Both the sub-range and enumerated types can be treated as basic types.

4. *Constructed* types: A constructed type is constructed from basic types and other basic types. Examples of constructed types are arrays, records and sets. Additionally, pointers and functions can also be treated as constructed types.

TYPE EXPRESSION:

It is an expression that denotes the type of an expression. The type of a language construct is denoted by a type expression

- Σ It is either a basic type or it is formed by applying operators called *type constructor* to other type expressions
- Σ A type constructor applied to a type expression is a type expression
- Σ A basic type is type expression

- *type error* : error during type checking
- *void* : no type value

The type of a language construct is denoted by a type expression. A type expression is either a basic type or is formed by applying an operator called a type constructor to other type expressions. Formally, a type expression is recursively defined as:

1. A basic type is a type expression. Among the basic types are *boolean* , *char* , *integer* , and *real* .A special basic type, *type_error* , is used to signal an error during type checking. Another special basic type is *void* which denotes "the absence of a value" and is used to check statements.
2. Since type expressions may be named, a type name is a type expression.
3. The result of applying a type constructor to a type expression is a type expression.
4. Type expressions may contain variables whose values are type expressions themselves.

TYPE CONSTRUCTORS: are used to define or construct the type of user defined types based on their dependent types.

Arrays : If T is a type expression and I is a range of integers, then *array* (*I* , *T*) is the type expression denoting the type of array with elements of type T and index set I.

For example, the Pascal declaration, var A: array[1 .. 10] of integer; associates the type expression ***array* (1..10, *integer*)** with A.

Products : If *T1* and *T2* are type expressions, then their Cartesian product ***T1 X T2*** is also a type expression.

Records : A record type constructor is applied to a tuple formed from field names and field types. For example, the declaration

Consider the declaration

```
type row = record
addr : integer;
lexeme : array [1 .. 15] of char
end;
var table: array [1 .. 10] of row;
```

The type row has type expression : **record ((addr x integer) x (lexeme x array(1 .. 15, char)))** and type expression of table is **array(1 .. 10, row)**

Note: Including the field names in the type expression allows us to define another record type with the same fields but with different names without being forced to equate the two.

Pointers: If T is a type expression, then ***pointer*** (***T***) is a type expression denoting the type "pointer to an object of type T".

For example, in Pascal, the declaration

var p: row declares variable p to have type ***pointer(* row *)***.

Functions : Analogous to mathematical functions, functions in programming languages may be defined as mapping a domain type D to a range type R. The type of such a function is denoted by the type expression D R. For example, the built-in function mod of Pascal has domain type int X int, and range type *int* . Thus we say mod has the type: **int x int -> int**

As another example, according to the Pascal declaration

function f(a, b: char) : integer;

Here the type of f is denoted by the type expression is **char x char pointer(integer)**

SPECIFICATIONS OF A TYPE CHECKER: Consider a language which consists of a sequence of declarations followed by a single expression

P ⟶ D ; E

D ⟶ D ; D | id : T

T ⟶ char | integer | array [num] of T | ^ T

E ⟶ literal | num | E mod E | E [E] | E ^

A **type checker** is a translation scheme that synthesizes the type of each expression from the types of its sub-expressions. Consider the above given grammar that generates programs consisting of a sequence of declarations D followed by a single expression E.

Specifications of a type checker for the language of the above grammar: A program generated by this grammar is

key : integer;
key mod 1999

Assumptions:

1. The language has three basic types: *char* , *int* and *type-error*

2. For simplicity, all arrays start at 1. For example, the declaration array[256] of char leads to the type expression *array* (1.. 256, char).

Rules for Symbol Table entry

D ⟶ id : T	addtype(id.entry, T.type)
T ⟶ char	T.type = char
T ⟶ integer	T.type = int
T ⟶ ^T_1	T.type = pointer(T_1 .type)
T ⟶ array [num] of T_1	T.type = array(1..num, T_1 .type)

TYPE CHECKING OF FUNCTIONS :

Consider the Syntax Directed Definition,

$E \rightarrow E_1 (E_2)$ E. type = if E_2 .type == s and

E_1 .type == s →t

then t

else type-error

The rules for the symbol table entry are specified above. These are basically the way in which the symbol table entries corresponding to the productions are done.

Type checking of functions

The production E -> E (E) where an expression is the application of one expression to another can be used to represent the application of a function to an argument. The rule for checking the type of a function application is

E -> E1 (E2) { *E.type* := if *E2.type* == s and *E1. type* == *s* -> *t* then *t* else *type_error* }

This rule says that in an expression formed by applying E1 to E2, the type of E1 must be a function *s* -> *t* from the type *s* of E2 to some range type *t* ; the type of E1 (E2) is *t* . The above rule can be generalized to functions with more than one argument byconstructing a product type consisting of the arguments. Thus n arguments of type *T1* , *T2*

... *Tn* can be viewed as a single argument of the type *T1 X T2 ... X Tn* . For example,

root : (real real) X real real

declares a function root that takes a function from reals to reals and a real as arguments and returns a real. The Pascal-like syntax for this declaration is

function root (function f (real) : real; x: real) : real

TYPE CHECKING FOR EXPRESSIONS: consider the following SDD for expressions

E →literal	E.type = char
E →num	E.type = integer
E →id	E.type = lookup(id.entry)
E →E_1 mod E_2	E.type = if E_1 .type == integer and E_2 .type==integer then integer

	else type_error
$E \rightarrow E_1 [E_2]$	E.type = if E_2 .type==integer and E_1 .type==array(s,t) then t
	else type_error
$E \rightarrow E_1$ ^	E.type = if E_1 .type==pointer(t) then t
	else type_error

To perform type checking of expressions, following rules are used. Where the synthesized attribute type for E gives the type expression assigned by the type system to the expression generated by E.

The following semantic rules say that constants represented by the tokens literal and num have type *char* and *integer* , respectively:

E -> literal { *E.type* := *char* }

E -> num { *E.type* := *integer* }

. The function *lookup* (*e*) is used to fetch the type saved in the symbol-table entry pointed to by e. When an identifier appears in an expression, its declared type is fetched and assigned to the attribute type:

E -> id { *E.type* := *lookup* (id . *entry*) }

. According to the following rule, the expression formed by applying the mod operator to two sub-expressions of type *integer* has type *integer* ; otherwise, its type is *type_error* .

E -> E1 mod E2 { *E.type* := if *E1.type* == *integer* and *E2.type* == *integer* then *integer* else *type_error* }

In an array reference E1 [E2], the index expression E2 must have type *integer* , inwhich case the result is the element type *t* obtained from the type *array* (*s, t*) of E1.

E -> E1 [E2] { *E.type* := if *E2.type* == *integer* and *E1.type* == *array* (*s, t*) then *t* else *type_error* }

Within expressions, the postfix operator yields the object pointed to by its operand. The type of E is the type *t* of the object pointed to by the pointer E:

E E1 { *E.type* := if *E1.type* == *pointer* (*t*) then *t* else *type_error* }

TYPE CHECKING OF STATEMENTS: Statements typically do not have values. Special basic type *void* can be assigned to them. Consider the SDD for the grammar below which generates Assignment statements conditional, and looping statements.

S →id := E	S.Type = if id.type == E.type then void else type_error
S →if E then S1	S.Type = if E.type == boolean then S1.type else type_error
S →while E do S1	S.Type = if E.type == boolean then S1.type else type_error
S →S1 ; S2	S.Type = if S1.type == void and S2.type == void then void else type_error

Since statements do not have values, the special basic type *void* is assigned to them, but if an error is detected within a statement, the type assigned to the statement is *type_error* .

The statements considered below are assignment, conditional, and while statements. Sequences of statements are separated by semi-colons. The productions given below can be combined with those given before if we change the production for a complete program to P -> D; S. The program now consists of declarations followed by statements.

Rules for type checking the statements are given below.

1. S id := E { *S.type* := if id . *type* == *E.type* then *void* else *type_error* }

This rule checks that the left and right sides of an assignment statement have the same type.

2. S if E then S1 { *S.type* := if *E.type* == *boolean* then *S1.type* else *type_error* }

This rule specifies that the expressions in an if -then statement must have the type *boolean* .

3. S while E do S1 { *S.type* := if *E.type* == *boolean* then *S1.type* else *type_error* }

This rule specifies that the expression in a while statement must have the type *boolean* .

4. S S1; S2 { *S.type* := if *S1.type* == *void* and *S2.type* == *void* then *void* else *type_error* }

Errors are propagated by this last rule because a sequence of statements has type *void* only if each sub-statement has type *void*.

IMPORTANT & EXPECTED QUESTIONS

1. What do you mean by THREE ADDRESS CODE? Generate the three-address code for the following code.

```
begin
        PROD: = 0;
        I: =1;
do
begin
        PROD:=PROD + A[I] B[I];
        I:=I+1;
End
```

```
while I <=20
end
```

2. Write a short note on Attributed grammar & Annotated parse tree.
3. Define an intermediate code form. Explain various intermediate code forms?
4. What is Syntax Directed Translation? Construct Syntax Directed Translation scheme to convert a given arithmetic expression into three address code.
5. What are Synthesized and Inherited attributes? Explain with examples?
6. Explain SDT for Simple Type checker?
7. Define and construct triples, quadruples and indirect triple notations of an expression: a * - (b + c).

ASSIGNMENT QUESTIONS:

1. Write Three address code for the below example

```
While( i<10)
{
a= b+c*-d;
i++;
}
```

2. What is a Syntax Directed Definition? Write Syntax Directed definition to convert binary value in to decimal?

SYMBOL TABLE

Symbol Table(ST) : Is a data structure used by the compiler to keep track of scope and binding information about names

- Symbol table is changed every time a name is encountered in the source;

Changes to table occur when ever a new name is discovered; new information about an existing name is discovered

As we know the compiler uses a symbol table to keep track of scope and binding information about names. It is filled after the AST is made by walking through the tree, discovering and assimilating information about the names. There should be two basic operations - to insert a new name or information into the symbol table as and when discovered and to efficiently lookup a name in the symbol table to retrieve its information.

Two common data structures used for the symbol table organization are -

1. Linear lists:- Simple to implement, Poor performance.
2. Hash tables:- Greater programming / space overhead, but, Good performance.

Ideally a compiler should be able to grow the symbol table dynamically, i.e., insert new entries or information as and when needed.

But if the size of the table is fixed in advance then (an array implementation for example), then the size must be big enough in advance to accommodate the largest possible program.

For each entry in declaration of a name

- The format need not be uniform because information depends upon the usage of the name
- Each entry is a record consisting of consecutive words
- To keep records uniform some entries may be outside the symbol table

Information is entered into symbol table at various times. For example,

- keywords are entered initially,
- identifier lexemes are entered by the lexical analyzer.

. Symbol table entry may be set up when role of name becomes clear , attribute values are filled in as information is available during the translation process.

For each declaration of a name, there is an entry in the symbol table. Different entries need to store different information because of the different contexts in which a name can occur. An entry corresponding to a particular name can be inserted into the symbol table at different stages depending on when the role of the name becomes clear. The various attributes that an entry in the symbol table can have are lexeme, type of name, size of storage and in case of functions - the parameter list etc.

A name may denote several objects in the same block

- int x; struct x { float y, z; }

The lexical analyzer returns the name itself and not pointer to symbol table entry. A record in the symbol table is created when role of the name becomes clear. In this case two symbol table entries are created.

Σ Aattributes of a name are entered in response to declarations

- Labels are often identified by colon

The syntax of procedure / function specifies that certain identifiers are formals, characters in a name. There is a distinction between token id, lexeme and attributes of the names. It is difficult to work with lexemes

- if there is modest upper bound on length then lexemes can be stored in symbol table
- if limit is large store lexemes separately

There might be multiple entries in the symbol table for the same name, all of them having different roles. It is quite intuitive that the symbol table entries have to be made only when the role of a particular name becomes clear. The lexical analyzer therefore just returns the name and not the symbol table entry as it cannot determine the context of that name. Attributes corresponding to the symbol table are entered for a name in response to the corresponding declaration. There has to be an upper limit for the length of the lexemes for them to be stored in the symbol table.

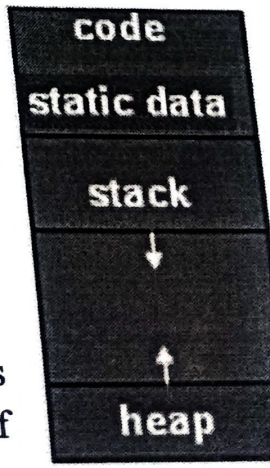
STORAGE ALLOCATION INFORMATION: Information about storage locations is kept in the symbol table.

If target code is assembly code, then assembler can take care of storage for various names and the compiler needs to generate data definitions to be appended to assembly code

If target code is machine code, then compiler does the allocation. No storage allocation is done for names whose storage is allocated at runtime

Information about the storage locations that will be bound to names at run time is kept in the symbol table. If the target is assembly code, the assembler can take care of storage for various names. All the compiler has to do is to scan the symbol table, after generating assembly code, and generate assembly language data definitions to be appended to the assembly language program for each name. If machine code is to be generated by the compiler, then the position of each data object relative to a fixed origin must be ascertained. The compiler has to do the allocation in this case. In the case of names whose storage is allocated on a stack or heap, the compiler does not allocate storage at all, it plans out the activation record for each procedure.

STORAGE ORGANIZATION: The runtime storage might be subdivided into :



- Target code ,
- Data objects,
- Stack to keep track of procedure activation, and
- Heap to keep all other information

This kind of organization of run-time storage is used for languages such as Fortran, Pascal and C. The size of the generated target code, as well as that of some of the data objects, is known at compile time. Thus, these can be stored

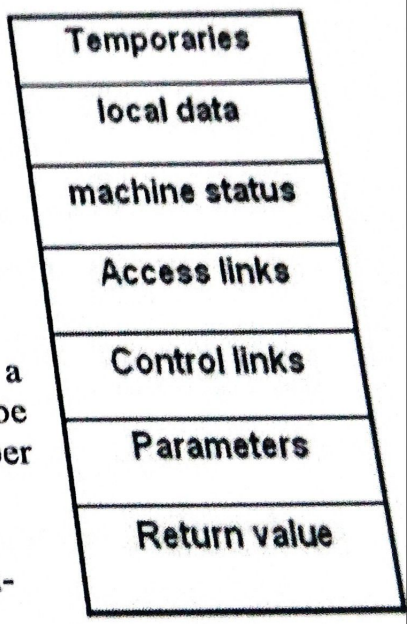
in statically determined areas in the memory.

STORAGE ALLOCATION PROCEDURE CALLS: Pascal and C use the stack for procedure activations. Whenever a procedure is called, execution of activation gets interrupted, and information about the machine state (like register values) is stored on the stack.

When the called procedure returns, the interrupted activation can be restarted after restoring the saved machine state. The heap may be used to store dynamically allocated data objects, and also other stuff such as activation information (in the case of languages where an activation tree cannot be used to represent lifetimes). Both the stack and the heap change in size during program execution, so they cannot be allocated a fixed amount of space. Generally they start from opposite ends of the memory and can grow as required, towards each other, until the space available has filled up.

ACTIVATION RECORD: An Activation Record is a data structure that is activated/ created when a procedure / function are invoked and it contains the following information about the function.

- Σ Temporaries: used in expression evaluation
- Σ Local data: field for local data
- Σ Saved machine status: holds info about machine status before procedure call
- Σ Access link : to access non local data
- Σ Control link : points to activation record of caller
- Σ Actual parameters: field to hold actual parameters
- Σ Returned value : field for holding value to be returned



The activation record is used to store the information required by a single procedure call. Not all the fields shown in the figure may be needed for all languages. The record structure can be modified as per the language/compiler requirements.

For Pascal and C, the activation record is generally stored on the run-time stack during the period when the procedure is executing.

Of the fields shown in the figure, access link and control link are optional (e.g. FORTR doesn't need access links). Also, actual parameters and return values are often stored in regi instead of the activation record, for greater efficiency.

- Σ The activation record for a procedure call is generated by the compiler. Generally, a field sizes can be determined at compile time.

However, this is not possible in the case of a procedure which has a local array whose size depends on a parameter. The strategies used for storage allocation in such cases will be discussed in forth coming lines.

STORAGE ALLOCATION STRATEGIES: The storage is allocated basically in the following THREE ways,

- Σ Static allocation: lays out storage at compile time for all data objects
- Σ Stack allocation: manages the runtime storage as a stack
- Σ Heap allocation :allocates and de-allocates storage as needed at runtime from heap

These represent the different storage-allocation strategies used in the distinct parts of the run-time memory organization (as shown in slide 8). We will now look at the possibility of using these strategies to allocate memory for activation records. Different languages use different strategies for this purpose. For example, old FORTRAN used static allocation, Algol type languages use stack allocation, and LISP type languages use heap allocation.

STATIC ALLOCATION: In this approach memory is allocated statically. So,Names are bound to storage as the program is compiled

- Σ No runtime support is required
- Σ Bindings do not change at run time
- Σ On every invocation of procedure names are bound to the same storage
- Σ Values of local names are retained across activations of a procedure

These are the fundamental characteristics of static allocation. Since name binding occurs during compilation, there is no need for a run-time support package. The retention of local name values across procedure activations means that when control returns to a procedure, the values of the locals are the same as they were when control last left. For example, suppose we had the following code, written in a language using static allocation:

```
function F( )
 {
        int a;
        print(a);
        a = 10;
 }
```

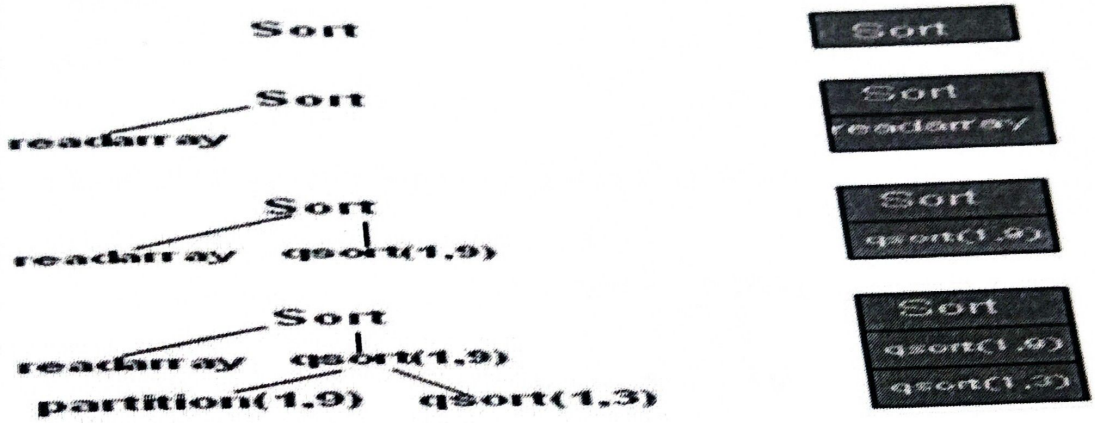
After calling F( ) once, if it was called a second time, the value of a would initially be 10, a this is what would get printed.

The type of a name determines its storage requirement. The address for this storage is an of from the procedure's activation record, and the compiler positions the records relative to target code and to one another (on some computers, it may be possible to leave this rela

position unspecified, and let the link editor link the activation records to the executable code) After this position has been decided, the addresses of the activation records, and hence of the storage for each name in the records, are fixed. Thus, at compile time, the addresses at which the target code can find the data it operates upon can be filled in. The addresses at which information is to be saved when a procedure call takes place are also known at compile time. Static allocation does have some limitations.

- Size of data objects, as well as any constraints on their positions in memory, must be available at compile time.
- No recursion, because all activations of a given procedure use the same bindings for local names.
- No dynamic data structures, since no mechanism is provided for run time storage allocation.

STACK ALLOCATION: Figure shows the activation records that are pushed onto and popped for the run time stack as the control flows through the given activation tree.



First the procedure is activated. Procedure readarray 's activation is pushed onto the stack, wh the control reaches the first line in the procedure sort . After the control returns from activation of the readarray , its activation is popped. In the activation of sort , the control t reaches a call of qsort with actuals 1 and 9 and an activation of qsort is pushed onto the top o stack. In the last stage the activations for partition (1,3) and qsort (1,0) have begun and e during the life time of qsort (1,3), so their activation records have come and gone from the leaving the activation record for qsort (1,3) on top.

CALLING SEQUENCES: A call sequence allocates an activation record and information into its field. A return sequence restores the state of the machine so that procedure can continue execution.

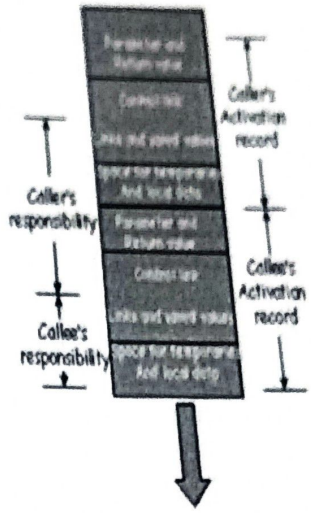
Calling sequence and activation records differ, even for the same language. The co calling sequence is often divided between the calling procedure and the procedure it cal

There is no exact division of runtime tasks between the caller and the colleen.

As shown in the figure, the register stack top points to the end of the machine status field in the activation record.

This position is known to the caller, so it can be made responsible for setting up stack top before control flows to the called procedure.

The code for the Callee can access its temporaries and the local data using offsets from stack top.



Call Sequence: In a call sequence, following sequence of operations is performed.

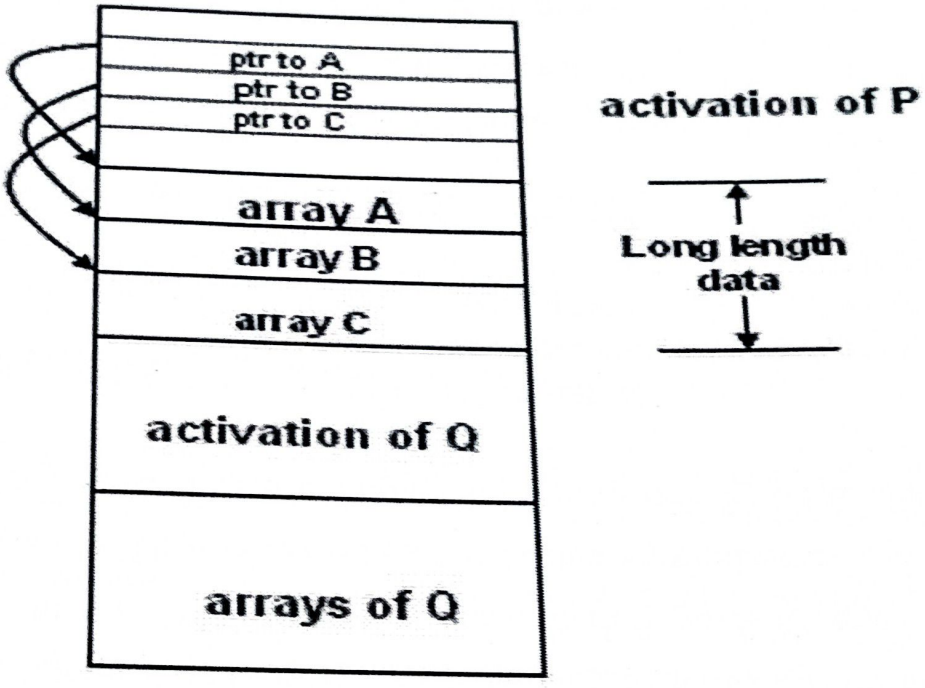
- Σ Caller evaluates the actual parameters
- Σ Caller stores return address and other values (control link) into callee's activation record
- Σ Callee saves register values and other status information
- Σ Callee initializes its local data and begins execution

The fields whose sizes are fixed early are placed in the middle. The decision of whether or not to use the control and access links is part of the design of the compiler, so these fields can be fixed at compiler construction time. If exactly the same amount of machine-status information is saved for each activation, then the same code can do the saving and restoring for all activations. The size of temporaries may not be known to the front end. Temporaries needed by the procedure may be reduced by careful code generation or optimization. This field is shown after that for the local data. The caller usually evaluates the parameters and communicates them to the activation record of the callee. In the runtime stack, the activation record of the caller i just below that for the callee. The fields for parameters and a potential return value are place next to the activation record of the caller. The caller can then access these fields using offse from the end of its own activation record. In particular, there is no reason for the caller to kno about the local data or temporaries of the callee.

Return Sequence : In a return sequence, following sequence of operations are performed.

1. Callee places a return value next to activation record of caller
2. Restores registers using information in status field
3. Branch to return address
4. Caller copies return value into its own activation record

As described earlier, in the runtime stack, the activation record of the caller is just below that for the callee. The fields for parameters and a potential return value are placed next to the activation record of the caller. The caller can then access these fields using offsets from the end of its own activation record. The caller copies the return value into its own activation record. In particular, there is no reason for the caller to know about the local data or temporaries of the callee. The given calling sequence allows the number of arguments of the called procedure to depend on the call. At compile time, the target code of the caller knows the number of arguments it is supplying to the callee. The caller knows the size of the parameter field. The target code of the called must be prepared to handle other calls as well, so it waits until it is called, then examines the parameter field. Information describing the parameters must be placed next to the status field so the callee can find it.



Long Length Data:

The procedure P has three local arrays. The storage for these arrays is not part of the activation record for P; only a pointer to the beginning of each array appears in the activation record. The relative addresses of these pointers are known at the compile time, so the target code can access array elements through the pointers. Also shown is the procedure Q called by P . The activation record for Q begins after the arrays of P. Access to data on the stack is through two pointers, top and stack top. The first of these marks the actual top of the stack; it points to the

position at which the next activation record begins. The second is used to find the local data. For consistency with the organization of the figure in slide 16, suppose the stack top points to the end of the machine status field. In this figure the stack top points to the end of this field in the activation record for Q. Within the field is a control link to the previous value of stack top when control was in calling activation of P. The code that repositions top and stack top can be generated at compile time, using the sizes of the fields in the activation record. When q returns, the new value of top is stack top minus the length of the machine status and the parameter fields in Q's activation record. This length is known at the compile time, at least to the caller. After adjusting top, the new value of stack top can be copied from the control link of Q.

Dangling References: Referring to locations which have been de-allocated.

```
void main()
{
   int *p;
   p = dangle(); /* dangling reference */
}

int *dangle();
{
   int i=23;
   return &i;
}
```

The problem of dangling references arises, whenever storage is de-allocated. A dangling reference occurs when there is a reference to storage that has been de-allocated. It is a logical error to use dangling references, since the value of de-allocated storage is undefined according to the semantics of most languages. Since that storage may later be allocated to another datum, mysterious bugs can appear in the programs with dangling references.

HEAP ALLOCATION: If a procedure wants to put a value that is to be used after its activation is over then we cannot use stack for that purpose. That is language like Pascal allows data to be allocated under program control. Also in certain language a called activation may outlive the caller procedure. In such a case last-in-first-out queue will not work and we will require a data structure like heap to store the activation. The last case is not true for those languages whose activation trees correctly depict the flow of control between procedures.

Limitations of Stack allocation: It cannot be used if,

- The values of the local variables must be retained when an activation ends
- A called activation outlives the caller

∑ In such a case de-allocation of activation record cannot occur in last-in first-out fashion
∑ Heap allocation gives out pieces of contiguous storage for activation records

There are two aspects of dynamic allocation -:

- Runtime allocation and de-allocation of data structures.
- Languages like Algol have dynamic data structures and it reserves some part of memory for it.

Initializing data-structures may require allocating memory but where to allocate this memory. After doing type inference we have to do storage allocation. It will allocate some chunk of bytes. But in language like LISP, it will try to give continuous chunk. The allocation in continuous bytes may lead to problem of fragmentation i.e. you may develop hole in process of allocation and de-allocation. Thus storage allocation of heap may lead us with many holes and fragmented memory which will make it hard to allocate continuous chunk of memory to requesting program. So,we have heap mangers which manage the free space and allocation and de-allocation of memory. It would be efficient to handle small activations and activations of predictable size as a special case as described in the next slide. The various allocation and de-allocation techniques used will be discussed later.
Fill a request of size s with block of size s ' where s ' is the smallest size greater than or equal to s

- For large blocks of storage use heap manager
- For large amount of storage computation may take some time to use up memory so that time taken by the manager may be negligible compared to the computation time

As mentioned earlier, for efficiency reasons we can handle small activations and activations of predictable size as a special case as follows:

1. For each size of interest, keep a linked list if free blocks of that size

2. If possible, fill a request for size s with a block of size s', where s' is the smallest size greate than or equal to s. When the block is eventually de-allocated, it is returned to the linked list came from.

3. For large blocks of storage use the heap manger.

Heap manger will dynamically allocate memory. This will come with a runt overhead. As heap manager will have to take care of defragmentation and garbage collec But since heap manger saves space otherwise we will have to fix size of activation at co time, runtime overhead is the price worth it.

ACCESS TO NON-LOCAL NAMES :

The scope rules of a language decide how to reference the non-local variables. There are t methods that are commonly used:

1. Static or Lexical scoping: It determines the declaration that applies to a name by exan the program text alone. E.g., Pascal, C and ADA.
2. Dynamic Scoping: It determines the declaration applicable to a name at run time, by considering the current activations. E.g., Lisp

ORGANIZATION FOR BLOCK STRUCTURES:

A block is a any sequence of operations or instructions that are used to perform a [sub] task. In any programming language,

- Σ Blocks contain its own local data structure.
- Σ Blocks can be nested and their starting and ends are marked by a delimiter.
- Σ They ensure that either block is independent of other or nested in another block. That is, it is not possible for two blocks B1 and B2 to overlap in such a way that first block B1 begins, then B2, but B1 end before B2.
- Σ This nesting property is called block structure. The scope of a declaration in a block-structured language is given by the most closely nested rule:

  1. The scope of a declaration in a block B includes B.

  2. If a name X is not declared in a block B, then an occurrence of X in B is in the scope of a declaration of X in an enclosing block B ' such that. B ' has a declaration of X, and. B ' is more closely nested around B then any other block with a declaration of X.

For example, consider the following code fragment.

```
main()
{                          BEGINNING of B0 ──────────────── Scope B0, B1, B3
   int a=0 ──────────────────────────────────────────────── Scope B0
   int b=0
   {                       BEGINNING of B1 ──────────────── Scope B1, B2
        int b=1
        {                  BEGINNING of B2 ──────────────── Scope B2
             int a=2
             print a, b
        }                  END of B2
                           ──────────────────────────────── Scope B3
        {                  BEGINNING of B3
             int b=3
             print a, b
        }                  END of B3
        print a, b
   }                       END of B1
   print a, b
}                          END of B0
```

For the example, in the above figure, the scope of declaration of b in B0 does not include B because b is re-declared in B1. We assume that variables are declared before the first stateme in which they are accessed. The scope of the variables will be as follows:

DECLARATION	SCOPE
int a=0	B0 not including B2
int b=0	B0 not including B1
int b=1	B1 not including B3
int a =2	B2 only
int b =3	B3 only

The outcome of the print statement will be, therefore:

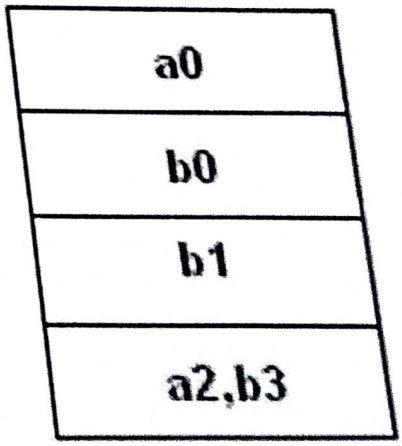
2 1
0 3
0 1
0 0

Blocks : . Blocks are simpler to handle than procedures

. Blocks can be treated as parameter less procedures

. Use stack for memory allocation

. Allocate space for complete procedure body at one time

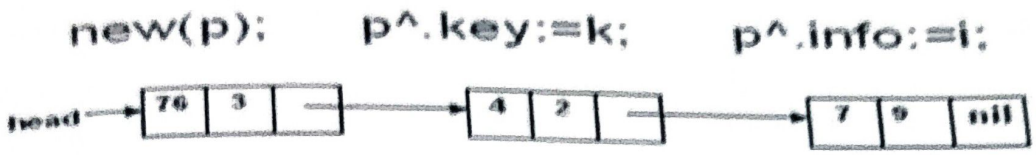


There are two methods of implementing block structure in compiler construction:

1. **STACK ALLOCATION:** This is based on the observation that scope of a declaration does not extend outside the block in which it appears, the space for declared name can be allocated when the block is entered and de-allocated when controls leave the block. The view treat block as a "parameter less procedure" called only from the point just before the block and returning only to the point just before the block.

2. **COMPLETE ALLOCATION:** Here you allocate the complete memory at one time. If there are blocks within the procedure, then allowance is made for the storage needed for declarations within the books. If two variables are never alive at the same time and are at same depth they ca be assigned same storage.

DYNAMIC STORAGE ALLOCATION:



Garbage : unreachable cells
- Lisp does garbage collection
- Pascal and C do not

head^.next := nil;

Dangling reference

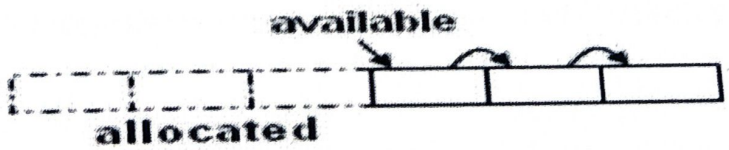
dispose(head^.next)

Generally languages like Lisp and ML which do not allow for explicit de-allocation of memory do garbage collection. A reference to a pointer that is no longer valid is called a 'dangling reference'. For example, consider this C code:

```
int main (void)
 {
        int* a=fun();
 }
int* fun()
 {
        int a=3;
        int* b=&a;
        return b;
 }
```

Here, the pointer returned by fun() no longer points to a valid address in memory as the activation of fun() has ended. This kind of situation is called a 'dangling reference'. In case of explicit allocation it is more likely to happen as the user can de-allocate any part of memory, even something that has to a pointer pointing to a valid piece of memory.

In Explicit Allocation of Fixed Sized Blocks , Link the blocks in a list , and Allocation and de-allocation can be done with very little overhead.



The simplest form of dynamic allocation involves blocks of a fixed size. By linking the blocks in a list, as shown in the figure, allocation and de-allocation can be done quickly with little or no storage overhead.

Explicit Allocation of Fixed Sized Blocks: In this approach, blocks are drawn from contiguous area of storage, and an area of each block is used as pointer to the next block

- The pointer available points to the first block
- Allocation means removing a block from the available list
- De-allocation means putting the block in the available list
- Compiler routines need not know the type of objects to be held in the blocks
- Each block is treated as a variant record

Suppose that blocks are to be drawn from a contiguous area of storage. Initialization of the area is done by using a portion of each block for a link to the next block. A pointer available points to the first block. Generally a list of free nodes and a list of allocated nodes is maintained, and whenever a new block has to be allocated, the block at the head of the free list is taken off and allocated (added to the list of allocated nodes). When a node has to be de-allocated, it is removed from the list of allocated nodes by changing the pointer to it in the list to point to the block previously pointed to by it, and then the removed block is added to the head of the list of free blocks. The compiler routines that manage blocks do not need to know the type of object that will be held in the block by the user program. These blocks can contain any type of data (i.e., they are used as generic memory locations by the compiler). We can treat each block as a variant record, with the compiler routines viewing the block as consisting of some other type. Thus, there is no space overhead because the user program can use the entire block for its own purposes. When the block is returned, then the compiler routines use some of the space from the block itself to link it into the list of available blocks, as shown in the figure in the last slide.

Explicit Allocation of Variable Size Blocks :

Limitations of Fixed sized block allocation: In explicit allocation of fixed size blocks, internal fragmentation can occur, that is, the heap may consist of alternate blocks that are free and in use, as shown in the figure.

The situation shown can occur if a program allocates five blocks and then de-allocates the second and the fourth, for example.

Fragmentation is of no consequence if blocks are of fixed size, but if they are of variable size, a situation like this is a problem, because we could not allocate a block larger than any one of the free blocks, even though the space is available in principle.

So, if variable- sized blocks are allocated, then internal fragmentation can be avoided, as we onl allocate as much space as we need in a block. But this creates the problem of externa fragmentation, where enough space is available in total for our requirements, but not enough

space is available in continuous memory locations, as needed for a block of allocated memory. For example, consider another case where we need to allocate 400 bytes of data for the next request, and the available continuous regions of memory that we have are of sizes 300, 200 and 100 bytes. So we have a total of 600 bytes, which is more than what we need. But still we are unable to allocate the memory as we do not have enough contiguous storage.

The amount of external fragmentation while allocating variable-sized blocks can become very high on using certain strategies for memory allocation.

So we try to use certain strategies for memory allocation, so that we can minimize memory wastage due to external fragmentation. These strategies are discussed in the next few lines.

. Storage can become fragmented, Situation may arise, If program allocates five blocks
. then de-allocates second and fourth block

IMPORTANT QUESTIONS:

1. What are calling sequence, and Return sequences? Explain briefly.
2. What is the main difference between Static & Dynamic storage allocation? Explain the problems associated with dynamic storage allocation schemes.
3. What is the need of a display associated with a procedure? Discuss the procedures for maintaining the display when the procedures are not passed as parameters.
4. Write notes on the static storage allocation strategy with example and discuss its limitations?
5. Discuss about the stack allocation strategy of runtime environment with an example?
6. Explain the concept of implicit de allocation of memory.
7. Give an example of creating dangling references and explain how garbage is created.

ASSIGNMENT QUESTIONS:

1. What is a calling sequence? Explain briefly.
2. Explain the problems associated with dynamic storage allocation schemes.
3. List and explain the entries of Activation Record.
4. Explain about parameter passing mechanisms.